

Duckiebot Command Cheat Sheet

SSH | Linux | scp | git | make | ROS2 Humble | tmux | Docker

Replace `duckieXX` with the number written on your robot. Replace `yournick` with your own nickname. Replace `team1` with your own repo folder.

EMERGENCY STOP

Robot running away? Open any shell in the container and run:

```
ros2 topic pub --once /duckieXX/wheels_cmd
  duckietown_msgs/msg/WheelsCmdStamped
  "{vel_left: 0.0, vel_right: 0.0}"
```

Ctrl+C stops the node you are looking at, nothing else.
space stops the robot while you are driving with the keyboard.

1 Connect to the robot

```
ssh duckie@duckieXX.local
```

Password: quackquack. XX = the number written on the robot.

VS Code shortcuts

mac: **Cmd** instead of **Ctrl**.

Ctrl+S — save; with SFTP this is what uploads the file to the robot

Ctrl+Shift+P — command palette (this is how you reach "SFTP: Config")

Ctrl+Shift+X — extensions

Ctrl+P — open a file by name

Ctrl+Shift+F — search across all files

Ctrl+G — go to line

Ctrl+L — select the current line

Ctrl+D — select the next occurrence of the same word

Ctrl+/** — comment or uncomment

Alt+Up / Alt+Down — move the line up or down

Ctrl+Shift+K — delete the line

F2 — rename everywhere

Ctrl+` — show or hide the built-in terminal

Ctrl+B — show or hide the sidebar

Ctrl+B in VS Code toggles the sidebar. Inside tmux, **Ctrl+B** is the tmux prefix. Different things.

2 SSH keys: stop typing the password

GENERATE — Windows PowerShell

```
ssh-keygen -t ed25519 -f .ssh/nickname -C
  "nickname"
```

GENERATE — mac / Linux

```
ssh-keygen -t ed25519 -f ~/.ssh/nickname -C
  nickname
```

COPY

```
scp .ssh/nickname.pub
  duckie@duckieXX.local:~/.ssh/nickname.pub
```

APPLY

```
ssh duckie@duckieXX.local "(echo; cat
  ~/.ssh/nickname.pub) >> ~/.ssh/authorized_keys
  && chmod 600 ~/.ssh/authorized_keys && rm
  ~/.ssh/nickname.pub"
```

CHECK — must log in with no password

```
ssh -i .ssh/nickname duckie@duckieXX.local
```

mac / Linux: `ssh-copy-id -i ~/.ssh/nickname.pub duckie@duckieXX.local` does COPY + APPLY in one command.

Only the `.pub` (public) file ever leaves your computer.

3 Linux shell basics

`pwd` — where am I

`ls` — list files here

`cd folder` — go into a folder

`cd ..` — go one folder up

`cat file` — print a file

`mkdir folder` — new folder

`touch file` — new empty file

`rm file` — delete a file

`rm -r folder` — delete a folder with everything in it

`chmod +x script.sh` — make a script runnable

`ls | grep .py` — filter the output, keep lines with `.py`

`nano file` — edit: save = **Ctrl+O**, **Enter**; exit = **Ctrl+X**

`micro file` — editor with normal **Ctrl+S** / **Ctrl+C** / **Ctrl+V**;
save = **Ctrl+S**, exit = **Ctrl+Q**

`micro` is not installed by default: add `micro` to

`requirements-apt.txt` and make `build`, or on the robot
itself `sudo apt install micro`.

Ctrl+C = stop the running program. **Tab** = complete the name. **arrow up** = previous command.

4 Copy files — run these ON YOUR COMPUTER

```
| scp myfile.py duckie@duckieXX.local:~/team1/
```

Send one file to the robot.

```
| scp -rp src/. duckie@duckieXX.local:~/team1/src/
```

Send a whole folder. The `/.` matters: without it you get `src/src` once the folder already exists there.

```
| scp 'duckie@duckieXX.local:~/team1/images/*.jpg'
| .
```

Get the pictures. The quotes matter: they stop your own shell expanding `*`.

```
| ./get_images.sh duckieXX
```

Same thing, script in the repo.

5 git

`git config --global user.name "Your Name"` — once, before your first commit

`git config --global user.email "you@example.com"` — once, same

`git clone <url>` — download the repo

`git pull` — get the newest changes

`git status` — what did I change

`git add .` — stage all changes

`git commit -m "message"` — save a snapshot

`git push` — upload your commits

Clone on the robot with the HTTPS url. You cannot push from the robot that way — push from your own computer.

git: history and branches

`git log --oneline` — history, one line per commit

`git diff` — show the changes line by line

`git branch` — list branches

`git switch -c feature-x` — create a branch and switch to it

`git switch main` — switch to an existing branch

`git merge feature-x` — merge a branch into the current one

`git stash` — put your changes aside so you can `git pull`

`git stash pop` — bring them back afterwards

Simplest rule: commit before you pull, then you never need stash.

.gitignore

`.gitignore` — one pattern per line; matching files are never committed

`echo ".env" >> .gitignore` — start ignoring a file

`git rm --cached .env` — stop tracking it, keeps the file on disk

`git commit -m "untrack .env"` — record it

`.gitignore` only affects files git is not already tracking: something already committed stays tracked until you `git rm --cached` it.

6 The project, on the robot

`export VEHICLE_NAME=duckieXX` — the robot's own name, part of every topic name

`export USER_NAME=yournick` — letters, digits and `_` only: no dashes, no Cyrillic

`make build` — build the image and the workspace

`make run` — start the container and open a shell in it

`make shell` — one more shell in the same container

`make down` — stop and delete the container

`make clean` — delete build/ install/ log/

After editing a `.py` or a launch file you do **not** rebuild — just restart the node.

`make build` only after adding or renaming files, or changing `setup.py` / `package.xml` / `requirements`.

7 Run the examples (inside the container)

`ros2 run blinker blinker.py` — LEDs cycle red, green, blue

`ros2 run camera camera.py` — saves pictures into images/

`ros2 run robot main.py` — movements: turns commands into wheel and LED commands

`ros2 run control_room main.py` — reads the arrow keys

`ros2 launch master_launch keyboard.launch.xml` — control_room + robot together

`ros2 launch master_launch demo.launch.xml` — the same, plus the camera

Driving keys (keyboard control)

`↑ / w` forward

`↓ / s` backward

`← / a` turn left

`→ / d` turn right

`space` stop

`1 2 3` LEDs red / green / blue

`0` LEDs white

`q` quit and stop the robot

8 Look inside ROS2

`ros2 topic list` — all topics on the network

`ros2 node list` — all running nodes

`ros2 topic info <topic>` — type of the topic, who publishes, who listens

`ros2 topic echo <topic>` — watch whatever is published

`ros2 topic echo --once <topic>` — one message instead of a flood

`ros2 topic pub <topic> <type> "<yaml>"` — publish repeatedly

`ros2 topic pub --once <topic> <type> "<yaml>"` — publish a single message

Try it in two panes, no robot needed

```
| pane 1: ros2 topic echo /hello
| pane 2: ros2 topic pub /hello
|         std_msgs/msg/String "{data: 'hi'}"
```

Robot topics (/duckieXX/...) and driving with no code

image/compressed— camera
range— distance sensor
tick— wheel encoders
wheels_cmd— wheel commands
led_pattern— LEDs
emergency_stop— emergency stop
temperature— temperature

Drive with no code at all

```
ros2 topic pub --once /duckieXX/wheels_cmd
  duckietown_msgs/msg/WheelsCmdStamped
  "{vel_left: 0.4, vel_right: 0.4}"
```

Wheel speeds go from -1.0 to 1.0; positive is forward, 0.0 stops.

All LEDs red with no code

```
ros2 topic pub --once /duckieXX/led_pattern
  duckietown_msgs/msg/LEDPattern "{rgb_vals:
  [{r: 1.0, a: 1.0}, {r: 1.0, a: 1.0}, {r: 1.0,
  a: 1.0}, {r: 1.0, a: 1.0}, {r: 1.0, a: 1.0}]}"
```

9 tmux: keep programs running after you disconnect

tmux new -s name— new session
tmux ls— list sessions
tmux attach -t name— go back into a session
tmux kill-session -t name— delete a session

tmux: press Ctrl+B, release, then

d— detach, everything keeps running
c— new window
n / p— next / previous window
<number>— go to that window
w— list windows
%— split left / right
"— split top / bottom
arrows— move between panes
[— scroll back through output: arrows and PgUp move, q exits
x then y— kill the PANE
& then y— kill the WINDOW

10 Docker

docker ps— running containers
docker ps -a— all containers, stopped ones too
docker images— images on this machine
docker logs -f <name>— follow the output of a container
docker exec -it <name> bash— shell inside a running container
docker start <name>— start a stopped container
docker stop <name>— stop it
docker rm -f <name>— delete it
docker run takes an image, docker start takes an existing container.

Your container is called duckiebot. The whole team shares it.

11 When it does not work

ros2: command not found
→ source /opt/ros/humble/setup.bash
Package 'blinker' not found
→ not built or not sourced: make build, then
source install/local_setup.bash
can't open file _local_setup_util_sh.py
→ source install/local_setup.bash — not
install/setup.bash
VEHICLE_NAME is not set
→ export it again: every new shell needs it
ros2 topic list shows nothing from the robot
→ use make run, not your own docker run
Edited a .py but nothing changed
→ restart the node. If it still looks old, it may be an SFTP problem: check the file content on the robot with cat
Nodes start, robot does not move
→ ros2 topic echo /duckieXX/wheels_cmd to see if commands arrive; check battery and emergency stop
LEDs flicker or ignore you
→ two nodes publish to led_pattern: stop blinker while driving